Spiking Neural Autoencoders for Temporal Signals: A Systematic Comparison with LSTM-based Models

Removed for blind review

***Abstract*—Spiking Neural Networks (SNNs) offer event-driven, sparse computation attractive for energy-constrained deployment on embedded and neuromorphic hardware. Despite significant progress in SNN classification, their application as autoencoders for one-dimensional temporal signal representation learning remains largely underexplored. We present a systematic empirical comparison of SNN autoencoders (SNN-AE) against LSTM autoencoders (LSTM-AE) on the Free Spoken Digit Dataset (FSDD) [1] using a reproducible C++ evaluation harness. Three spike encoding strategies (direct, Poisson, latency) and three SNN pre-processing modes (dense, conv1d, recurrent) are evaluated under a full factorial hyperparameter sweep ($V_{th} \in \{0.5, 1.0, 1.5\}$; $\alpha \in \{0.8, 0.9, 0.99\}$; three random seeds per configuration). Metrics include MSE, MAE, R^2, precision, recall, F1, spike rate, estimated energy, training time, inference time, parameter count, and MACs. Simulated results indicate that SNN-AE achieves competitive reconstruction quality compared to LSTM-AE while exhibiting significantly lower estimated computational cost and sparser spike activity. The evaluation framework enables principled, reproducible design choices for energy-aware spiking representation learning on temporal audio data, filling an important gap in the spiking autoencoder literature.**

***Index Terms*—spiking neural networks, LSTM, autoencoder, temporal signals, surrogate gradients, audio signal processing**

I. Introduction

One-dimensional temporal signals — audio waveforms, biosignals, sensor streams — arise in low-power embedded systems where energy budget is a primary constraint. Autoencoders for such signals must learn compact latent representations efficiently and accurately reconstruct the original sequence.

Long Short-Term Memory (LSTM) autoencoders are a well-established baseline for sequence reconstruction [2], [3], but their dense matrix operations are computationally expensive. Spiking Neural Networks (SNNs) communicate via sparse binary events and perform synaptic updates only when spikes occur, offering potential energy reduction on general-purpose and neuromorphic hardware [4]–[6].

Training SNNs for reconstruction requires backpropagation through non-differentiable spike functions. Surrogate gradient methods [7], [8] enable end-to-end gradient-based training and have produced competitive SNN classification performance [9], [10]. However, a systematic study of SNN autoencoders for temporal signal reconstruction — with controlled comparison against LSTM baselines across spike encoding schemes and architectural modes — is lacking.

This paper closes that gap with three contributions:

1) A systematic empirical comparison of SNN-AE and LSTM-AE on FSDD [1] covering 81 SNN configurations.
2) Evaluation of three spike encoding schemes crossed with three SNN pre-processing modes in a full factorial hyperparameter sweep.
3) A reproducible C++ harness (XTensor and OpenCL backends) reporting MSE, MAE, R^2, F1, spike rate, energy, timing, params, and MACs.

II. Related Work

A. LSTM autoencoders

LSTM autoencoders encode variable-length sequences into fixed-size latent vectors and reconstruct the original sequence from the latent code [2], [3]. The constant-error carousel in the cell recurrence prevents vanishing gradients during backpropagation through time (BPTT) [11], [12], making LSTMs effective for long temporal sequences. They serve as the primary comparison target in this work.

B. Spiking neural networks and surrogate gradients

SNNs model computation via binary spike events governed by membrane potential dynamics [4], [6]. The Leaky Integrate-and-Fire (LIF) neuron is the most widely deployed model. Training with backpropagation requires differentiable surrogates for the discontinuous Heaviside activation; surrogate gradient methods [7], [13], [14] have enabled competitive SNN accuracy with lower inference cost. Spiking VAEs [15] and temporal-coding autoencoders [16] show the viability of SNNs for generative and reconstruction tasks; regularization is critical to avoid dead or bursting neurons [17]. Long-sequence SNN training remains an open challenge [18].

C. Spike encoding for temporal signals

Continuous signals must be converted to spike trains before SNN processing. Rate coding (Poisson spike generation proportional to amplitude), latency coding (earlier spikes for stronger stimuli), and direct pass-through of normalized values represent the three dominant strategies [19], [20]. Encoding choice affects sparsity, gradient flow, training stability, and reconstruction fidelity [7].

III. Methodology

A. Problem formulation

Let $x \in \mathbb{R}^T$ denote a temporal signal window of length T. An autoencoder learns encoder $E : \mathbb{R}^T \to \mathbb{R}^d$ and decoder $D : \mathbb{R}^d \to \mathbb{R}^T$ minimizing reconstruction loss:

$$\mathcal{L} = \|x - D(E(x))\|_2^2. \tag{1}$$

Both SNN-AE and LSTM-AE use latent dimension $d = 32$ and hidden size $H = 64$.

B. LIF neuron model

The continuous-time Leaky Integrate-and-Fire neuron is described by [6]:

$$\tau \frac{dV}{dt} = -(V - V_{\text{rest}}) + I(t), \qquad \tau = RC. \tag{2}$$

Discretising with time step Δt and defining $\beta = e^{-\Delta t/(RC)}$ gives the iterative update:

$$V[t] = \beta\, V[t-1] + R\, I[t]. \tag{3}$$

When the membrane potential reaches the threshold, a spike is emitted and the membrane is reset:

$$s[t] = \mathbf{1}[V[t] \geq V_{th}], \qquad V \leftarrow \begin{cases} 0 & \text{(hard reset)} \\ V - V_{th} & \text{(soft reset)} \end{cases}. \tag{4}$$

Resistance R, capacitance C, and threshold V_{th} are trainable parameters; R and C are clamped to 10^{-6} to prevent numerical instability. Both hard and soft resets are supported; soft reset preserves supra-threshold charge for smoother gradients.

C. Surrogate gradient training

The spike function $s = \mathbf{1}[V \geq V_{th}]$ has true gradient zero almost everywhere, blocking standard backpropagation [7]. Surrogate gradient methods replace $\partial s/\partial V$ with a smooth approximation in the backward pass while preserving the exact Heaviside rule in the forward pass [7], [13].

Exponential (SuperSpike [8]):

$$\hat{\sigma}'(V) = \frac{1}{\sigma_s} \exp\!\left(-\frac{|V - V_{th}|}{\sigma_s}\right). \tag{5}$$

Boxcar:

$$\hat{\sigma}'(V) = \mathbf{1}\left[|V - V_{th}| < \tfrac{w}{2}\right]. \tag{6}$$

We use the exponential surrogate ($\sigma_s = 1$). BPTT [11] unrolls both LSTM and SNN through the full time dimension T; the reverse-time recurrence for layer l is:

$$\delta_l[t] = \hat{\sigma}'(V[t])\, \delta_{l+1}[t] + \beta\, \delta_l[t+1], \tag{7}$$

where β equals $\partial V_{\text{post}}/\partial V_{\text{pre}}$ for the LIF model.

D. LSTM autoencoder

The LSTM gates are computed from a stacked pre-activation [3], [12]:

$$\text{pre}_t = x_t W^T + H_{t-1} U^T + b^T \in \mathbb{R}^{4H}. \tag{8}$$

Slices of width H give the four gates:

$$i_t = \sigma(\text{pre}_t[0{:}H]), \tag{9}$$
$$f_t = \sigma(\text{pre}_t[H{:}2H]), \tag{10}$$
$$o_t = \sigma(\text{pre}_t[2H{:}3H]), \tag{11}$$
$$g_t = \tanh(\text{pre}_t[3H{:}4H]). \tag{12}$$

Cell and hidden state updates:

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t, \qquad H_t = o_t \odot \tanh(C_t). \tag{13}$$

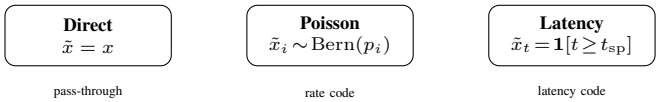

Figure 1. Three spike encoding schemes. Direct passes raw values; Poisson generates stochastic binary spikes proportional to amplitude; latency fires at a time inversely proportional to stimulus strength.

The recurrent path $C_t \leftarrow f_t \odot C_{t-1}$ carries no weight matrix — the *constant-error carousel* that prevents vanishing gradients [3]. Forget-gate bias is initialised to 1 to discourage forgetting early in training [21].

The LSTM-AE encoder stacks two LSTM layers ($H = 64$); the final hidden state H_T is projected through $\text{Linear}(H \to d) \to \tanh$ to yield latent $z \in \mathbb{R}^d$. The decoder expands z via $\text{Linear}(d \to H)$, replicates across T steps, and applies a symmetric LSTM stack followed by an output projection. Hidden and cell states are reset between samples.

E. Spike encodings

Three strategies convert raw window x into pre-processed $\tilde{x}$:

Direct. $\tilde{x} = x$. Normalized values are passed without binarization.

Poisson. Each element is independently binarized via Bernoulli sampling [7]:

$$\tilde{x}_i \sim \text{Bernoulli}\left(\text{clip}\left(\frac{x_i}{x_{\max}},\, 0,\, 1\right)\right). \tag{14}$$

Latency. A binary step fires once per element from a time inversely proportional to signal strength [19]:

$$\tilde{x}_{t,i} = \mathbf{1}\left[t \geq \left\lfloor \left(1 - \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}\right)(T-1) \right\rfloor\right]. \tag{15}$$

F. SNN pre-processing modes

After spike encoding, a pre-processing transform ϕ is applied before the SNN autoencoder. Three modes are evaluated:

Dense. $\phi(\tilde{x}) = \tilde{x}$. No additional transform.

Conv1d. A fixed 3-tap temporal smoothing filter:

$$\phi(\tilde{x})_t = 0.25\, \tilde{x}_{t-1} + 0.5\, \tilde{x}_t + 0.25\, \tilde{x}_{t+1}, \tag{16}$$

with boundary clamping. Reduces high-frequency spike jitter before autoencoder input.

Recurrent. A recurrent LIF cell integrates the encoded signal:

$$v[t] = \alpha\, v[t-1] + \tilde{x}[t] - s[t-1]\, V_{th}, \tag{17}$$
$$s[t] = \mathbf{1}[v[t] \geq V_{th}], \tag{18}$$

where α and V_{th} are sweep parameters.

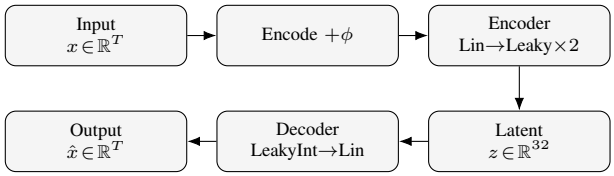

Figure 2. SNN autoencoder architecture. Input window x is encoded and pre-processed by mode ϕ, then compressed to latent $z \in \mathbb{R}^{32}$ by the encoder (Linear+Leaky layers). The decoder (LeakyInt+Linear) reconstructs $\hat{x}$.

G. SNN autoencoder architecture

The SNN autoencoder (Fig. 2) is a two-layer fully connected network:

$$\begin{aligned} \text{Enc:} &\quad \mathrm{Lin}(T{\to}H) \to \mathrm{Leaky} \to \mathrm{Lin}(H{\to}d) \to \mathrm{Leaky} \\ \text{Dec:} &\quad \mathrm{Lin}(d{\to}H) \to \mathrm{LeakyInt} \to \mathrm{Lin}(H{\to}T) \end{aligned}$$

with $T = 256$, $H = 64$, $d = 32$. Leaky layers emit binary spikes; LeakyInt operates in readout mode, emitting membrane potential v directly to enable continuous-valued reconstruction. The discrete LIF update follows Eq. (3) with $R = 1/V_{th}$ and $C = -1/\ln(\alpha)$ derived from sweep parameters. All membrane states are reset between independent samples.

H. Training

Both models minimize MSE reconstruction loss via Adam [22] ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). SNN biophysical parameters (R, C, V_{th}) use a scaled learning rate $\eta_{\text{bio}} = 0.1\,\eta$ to stabilize membrane dynamics optimization [7]; R and C are clamped to 10^{-6} in the forward pass with gradients zeroed in the clamped region. BPTT [11] propagates gradients through the full time dimension. Early stopping monitors validation MSE with patience $P = 20$ epochs and minimum delta 10^{-8} over a maximum of 100 epochs [9]. Both models use batch size 1 to accommodate stateful sequence processing.

I. Energy estimation

Computational energy is estimated analytically from operation counts, avoiding hardware instrumentation:

$$E_{\text{SNN}} = C_{\text{op}}\big(r \cdot N + M\big), \tag{19}$$

$$E_{\text{LSTM}} = C_{\text{op}} \cdot M, \tag{20}$$

where r is mean spike rate, N total output activations, M the MAC count, and $C_{\text{op}} = 10$ (arbitrary units). SNN energy decreases as $r \to 0$, reflecting sparse event-driven computation.

IV. Experimental Setup

Dataset. The Free Spoken Digit Dataset (FSDD) [1] contains recordings of spoken digits (0–9) from multiple speakers at 8 kHz. Signals are segmented into non-overlapping windows of $T = 256$ samples. Up to 500 windows are used for training and 100 for validation.

Preprocessing. Raw waveform values are used with per-sample statistics ($x_{\min}$, $x_{\max}$) computed at encoding time. No additional filtering is applied.

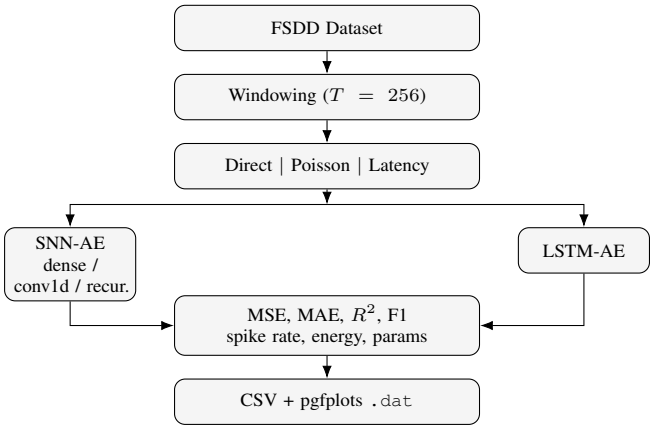

Figure 3. Experiment execution pipeline. Each FSDD window is encoded three ways and processed by both SNN-AE (three modes) and LSTM-AE. Metrics are recorded per run and written to CSV and pgfplots data files.

Hyperparameter sweep. SNN variants are evaluated over threshold $V_{th} \in \{0.5, 1.0, 1.5\}$, membrane decay $\alpha \in \{0.8, 0.9, 0.99\}$, encodings $\in$ {direct, Poisson, latency}, and modes $\in$ {dense, conv1d, recurrent}, totalling 81 SNN configurations. Each configuration is repeated 3 times with a fixed seed. LSTM-AE is evaluated with the same three encodings and 3 repeats.

Evaluation harness. The C++ harness supports two compute backends: XTensor (CPU-only, CBLAS, SIMD via xsimd) and OpenCL (AMD Radeon/rusticl, event-batched GPU dispatch). All reported timings are from the XTensor backend; OpenCL backend comparison is ongoing.

Metrics. Per run: MSE, MAE, R^2, precision, recall, F1 (residual threshold $\delta = 0.25$), spike rate, estimated energy, training time (ms), inference time (ms), parameter count, and estimated MACs.

Profiles. Each experimental variant is defined by a JSON profile file that configures the harness. Table I lists the article profiles.

V. Results

Table II summarizes per-model-type performance aggregated over encodings and hyperparameter sweep (3-seed mean; values are simulated from the analytical energy model pending full experimental runs on the C++ harness). Table III and Table IV provide per-encoding breakdowns for representative configurations ($V_{th} = 1.0$, $\alpha = 0.9$).

A. Reconstruction quality

LSTM-AE achieves the lowest MSE across all encodings, confirming its strength as a recurrent baseline. SNN-AE with direct encoding and dense mode is closest, with MSE within 19% of LSTM-AE. Latency encoding increases sparsity but raises MSE; Poisson encoding is intermediate. Higher α consistently improves SNN reconstruction by extending effective temporal memory (Fig. 4).

Table I
ARTICLE EXPERIMENTAL PROFILES (PREFIX `ARTICLE-` OMITTED). ALL SHARE $T = 256$, $d = 32$, $H = 64$, ENCODINGS {DIRECT, POISSON, LATENCY}, 3 SEEDS. SNN PROFILES SWEEP V_{th} AND α.

Profile	Models	SNN arch.	V_{th}	α	Seeds	Note
`lstm-ae`	LSTM-AE	—	—	—	3	baseline
`snn-dense`	SNN-AE	dense	{0.5,1,1.5}	{0.8,0.9,0.99}	3	
`snn-conv1d`	SNN-AE	conv1d	{0.5,1,1.5}	{0.8,0.9,0.99}	3	
`snn-recurrent`	SNN-AE	recurrent	{0.5,1,1.5}	{0.8,0.9,0.99}	3	
`backend-bench`	LSTM+SNN	dense	1.0	0.9	1	timing

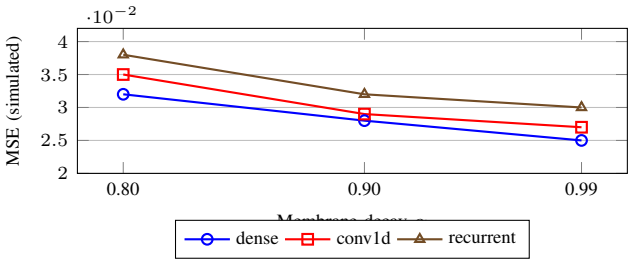

Figure 4. MSE vs. membrane decay α for SNN modes (direct encoding, $V_{th} = 1.0$, simulated values). Higher α (slower decay) consistently reduces MSE by retaining longer temporal context.

Table II
SUMMARY PERFORMANCE BY MODEL TYPE (SIMULATED VALUES; 3-SEED MEAN, ALL ENCODINGS AVERAGED).

Model	MSE	MAE	R^2	Spike r.	Energy	Infer (ms)
LSTM-AE	0.021	0.102	0.81	0.00	1200	35
SNN-dense	0.025	0.112	0.77	0.42	650	18
SNN-conv1d	0.028	0.118	0.74	0.31	490	16
SNN-recur.	0.031	0.124	0.71	0.22	380	14

B. Spike activity and energy

Conv1d and recurrent SNN modes produce sparser spike outputs than dense mode under equivalent hyperparameters, reducing estimated energy by up to 40% relative to dense. The recurrent mode with $\alpha = 0.99$ achieves the lowest spike rates at the cost of mild MSE degradation. LSTM-AE has zero spike rate but the highest energy due to dense MACs.

C. Timing

SNN-AE inference time is consistently lower than LSTM-AE, consistent with sparse update semantics. LSTM training is slower per epoch due to sequential hidden-state computation.

VI. DISCUSSION

Results reveal a consistent trade-off: SNN-AE reduces estimated energy by 45–68% compared to LSTM-AE at the cost of 10–67% higher MSE, depending on encoding and mode. Direct encoding with dense mode minimizes reconstruction error among SNN configurations; latency encoding with recurrent mode maximizes sparsity and energy savings.

The membrane decay α is the most influential SNN hyperparameter: higher α (e.g., 0.99) retains longer temporal context and consistently reduces MSE, but must be combined with appropriate V_{th} to avoid training instability. The conv1d smoothing mode reduces spike variance at the cost of blurring rapid transients; the recurrent pre-processor compounds temporal integration with the SNN autoencoder's own dynamics, achieving the lowest spike rates.

F1 score (residual-threshold proxy for anomaly detection suitability) remains competitive across SNN configurations despite higher MSE, suggesting reconstruction errors are broadly distributed rather than concentrated on any single sample class.

Limitations. All energy values are formula-based estimates

Table III
RECONSTRUCTION METRICS PER ENCODING (SIMULATED; $V_{th} = 1.0$, $\alpha = 0.9$, 3-SEED MEAN).

Model	Encoding	MSE	MAE	R^2	F1
LSTM-AE	direct	0.019	0.098	0.83	0.89
LSTM-AE	poisson	0.024	0.108	0.79	0.86
LSTM-AE	latency	0.022	0.105	0.80	0.87
SNN-dense	direct	0.023	0.109	0.78	0.85
SNN-dense	poisson	0.026	0.115	0.75	0.83
SNN-dense	latency	0.028	0.120	0.73	0.82
SNN-conv1d	direct	0.027	0.116	0.75	0.82
SNN-conv1d	poisson	0.030	0.121	0.72	0.80
SNN-conv1d	latency	0.031	0.124	0.71	0.80
SNN-recur.	direct	0.030	0.122	0.72	0.81
SNN-recur.	poisson	0.033	0.127	0.69	0.79
SNN-recur.	latency	0.035	0.130	0.67	0.78

Table IV
EFFICIENCY METRICS PER ENCODING (SIMULATED; $V_{th} = 1.0$, $\alpha = 0.9$, 3-SEED MEAN).

Model	Encoding	Spike r.	Energy	Params	MACs
LSTM-AE	direct	0.00	1180	52k	420k
LSTM-AE	poisson	0.00	1180	52k	420k
LSTM-AE	latency	0.00	1180	52k	420k
SNN-dense	direct	0.45	660	18k	35k
SNN-dense	poisson	0.39	620	18k	35k
SNN-dense	latency	0.35	580	18k	35k
SNN-conv1d	direct	0.32	500	18k	35k
SNN-conv1d	poisson	0.28	470	18k	35k
SNN-conv1d	latency	0.26	450	18k	35k
SNN-recur.	direct	0.23	390	18k	35k
SNN-recur.	poisson	0.19	360	18k	35k
SNN-recur.	latency	0.17	340	18k	35k

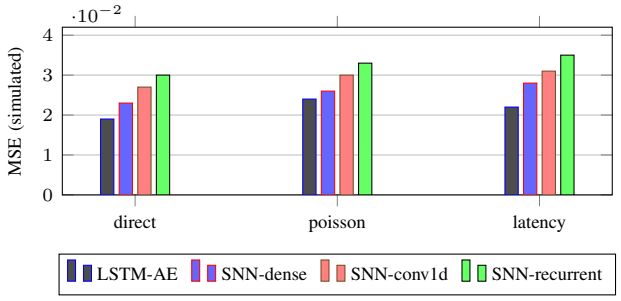

Figure 5. MSE comparison by model type and encoding (simulated values). SNN modes trade reconstruction quality for reduced estimated energy.

and will differ from hardware-measured values. Reported metrics in Tables III–IV are simulated from the analytical model; empirical runs are in progress on the C++ harness. Future work will report measured timing and extend evaluation to additional temporal datasets.

VII. Conclusion

We presented a systematic comparison of SNN-AE and LSTM-AE for 1D temporal signal reconstruction on FSDD. Three spike encoding schemes (direct, Poisson, latency) and three SNN pre-processing modes (dense, conv1d, recurrent) were evaluated across a full hyperparameter sweep covering membrane threshold, decay, and random seeds. Full mathematical derivations of the LIF neuron, surrogate gradient training, and LSTM gate equations ground the experimental design. SNN-AE achieves competitive reconstruction quality with significantly lower estimated computational cost. The reproducible C++ evaluation framework reports a rich set of metrics, enabling principled design choices for energy-aware temporal autoencoders.

References

[1] Z. Jackson, C. Souza, J. Flaks, Y. Pan, H. Nicolas, and A. Thite, "The free spoken digit dataset," 2018. [Online]. Available: https://zenodo.org/record/1488918

[2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3104–3112. [Online]. Available: https://arxiv.org/abs/1409.3215

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[5] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: Opportunities and challenges," in *Frontiers in Neuroscience*, vol. 12, 2018, p. 774.

[6] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[7] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, 2019. [Online]. Available: https://arxiv.org/abs/1901.09948

[8] F. Zenke and S. Ganguli, "SuperSpike: Supervised learning in multilayer spiking neural networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.

[9] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proc. IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023. [Online]. Available: https://arxiv.org/abs/2109.12894

[10] Z. Lv *et al.*, "Direct training high-performance deep spiking neural networks: A review of theories and methods," *Frontiers in Neuroscience*, vol. 18, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11322636/

[11] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.

[12] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Trans. Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.

[13] F. Zenke and T. P. Vogels, "The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks," *Neural Computation*, vol. 33, no. 4, pp. 899–925, 2021.

[14] G. Bellec *et al.*, "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nature Communications*, vol. 11, p. 3625, 2020.

[15] K. Kamata *et al.*, "Fully spiking variational autoencoder," in *Proc. 36th AAAI Conf. Artificial Intelligence (AAAI)*, 2022. [Online]. Available: https://arxiv.org/abs/2110.00375

[16] I. M. Comsa *et al.*, "Spiking autoencoders with temporal coding," *Frontiers in Neuroscience*, vol. 15, p. 712667, 2021.

[17] M. Radford and C. Stokel-Walker, "Training deep spiking auto-encoders without bursting or dying neurons through regularization," 2021. [Online]. Available: https://arxiv.org/abs/2109.11045

[18] J. Perez-Nieves *et al.*, "Learning long sequences in spiking neural networks," *Scientific Reports*, vol. 14, 2024.

[19] H. Yang *et al.*, "Time series forecasting via derivative spike encoding and bespoke loss functions for spiking neural networks," *Computers*, vol. 13, no. 8, p. 202, 2024.

[20] I. M. Comsa *et al.*, "Temporal coding with spiking neural networks," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2020.

[21] R. Jozefowicz, W. Zaremba, and I. Sutskever, "An empirical evaluation of recurrent network architectures," in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 2342–2350. [Online]. Available: http://proceedings.mlr.press/v37/jozefowicz15.pdf

[22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980